Data Preparation and Preprocessing

Note: I made this process per block of code based on it's functionality so it won't be too overwhleming to look at when reviewing.

(i) Purpose of Each Line of Code

- `import pandas as pd`, `import numpy as np`, `from sklearn.preprocessing import MinMaxScaler`
  - The following statements import the essential libraries for the project with Pandas taking care of structured data tables, Numpy dealing with advanced mathematics in terms of array manipulation, and Scikit-Learn offering the scaling facilities required for the normalization of models.
- `df = pd.read_csv('/Sample_ Superstore.csv')`
  - This line performs the vital function of accessing the unprocessed .csv stored externally, then importing this data into an organized DataFrame object, which serves as the foundational structure for the analytical tasks.
- `df = df.drop_duplicates()`
  - This command examines each row within the dataset to find and delete perfectly identical records. This is crucial as duplicate records can create a false perception of certain patterns, resulting in a model that is overfitted or biased.
- `df.loc[df['Sales'] < 0, 'Sales'] = np.nan`, `df.loc[df['Quantity'] <= 0, 'Quantity'] = np.nan`
  - This line applies a data validation rule that looks for logistically impossible negative entries in the Sales column and replaces them with a null value for the model not to learn from false information.
- `df['Sales'] = df['Sales'].fillna(df['Sales'].median())`, `df['Profit'] = df['Profit'].fillna(df['Profit'].median())`, `df['Quantity'] =`

`df['Quantity'].fillna(df['Quantity'].median())`, `df['Discount'] = df['Discount'].fillna(df['Discount'].median())`

- This line executes data imputation by determining the median of the Sales column and filling it to the empty values. This helps us retain the row without causing any extreme statistical outliers.

- `df['Segment'] = df['Segment'].str.strip().str.capitalize()`, `df['Category'] = df['Category'].str.strip().str.capitalize()`
  - This command will remove invisible spaces before and after the text and will also standardize the case of the text so that 'Consumer' won't be considered a different business category than ' consumer' as the computer sees both as different.
- `region_tax_data = pd.DataFrame({'Region': ['Central', 'South', 'East', 'West'],'Tax_Rate': [0.05, 0.07, 0.06, 0.08]})`
  - Here, we create another new secondary DataFrame from scratch that includes tax information for different regions as another external knowledge that deepens our original Superstore dataset.
- `df = pd.merge(df, region_tax_data, on='Region', how='left')`
  - This line adds the tax rates to each individual transaction based on the transaction's region. This is similar to how joins work in relational databases, and further extends our features / variables to improve predictions.
- `corr = df[['Sales', 'Quantity', 'Discount', 'Profit']].corr()`, `print("Correlation Matrix:\n", corr['Profit'])`
  - This command creates a matrix that calculates linear correlations of variables which is essential to determine which particular feature drives our target, Profit, the most.
- `scaler = MinMaxScaler()`
  - This starts the scaling object that has been designed to transform the data so that all numerical values fall within the same range, generally between 0 and 1, giving the model, a neutral perspective.
- `feature_cols = ['Sales', 'Quantity', 'Discount', 'Tax_Rate']`, `df[feature_cols] = scaler.fit_transform(df[feature_cols])`
  - This line computes the scaling of each column by determining the minimum and maximum values of each column thus allowing highly valued variables like

Sales to not numerically dominate smaller valued variables like Discount.

- `df = pd.get_dummies(df, columns=['Segment', 'Category', 'Region'])`
  - This line does what is called "One-Hot Encoding." It turns the descriptive text categories into a set of binary columns (0's and 1's) so the categorical data can be processed by the linear regression math algorithms.
- `df = df[(df['Profit'] > -500) & (df['Profit'] < 1000)]`
  - This critical filtering line removes extreme financial outliers where the company experienced massive losses or rare massive gains, ensuring the linear model focuses on standard business transactions.
- `df = df[df['Sales'] < 0.4]`
  - By capping the scaled sales value, this line prevents "leverage points" from skewing the slope of the regression line, which helps in maintaining a more accurate and representative trend for the majority of the data.
- `df_final = df.drop(columns=[...])`
  - This command is now placed after the filtering logic to ensure that the final exported dataset contains only the relevant features and rows that have passed all cleaning and filtering criteria.
- `df_final.to_csv('cleaned_sample_superstore.csv', index=False)`
  - This line exports the high-quality, filtered data into a persistent file format, allowing subsequent cells to import a dataset that is mathematically optimized for regression without extreme noise.
- `print("\nData Preparation Complete. Final shape:", df_final.shape)`
  - This diagnostic output provides a final confirmation of the row count (9,902), allowing the analyst to verify exactly how many outlier records were removed during the cleaning process.

(ii) Input

- **Raw Data**: The input for this cell is the original `Sample_Superstore.csv` file. It contains raw transactional data including sales, quantity, discounts, profit, and categorical labels like segment and region.

- **Predefined Tax Mapping**: A secondary input, a manually defined, dictionary-style DataFrame, provides the tax rates (5% to 8%) associated with the four main geographical areas in the dataset.
- **Library Parameters**: The code makes use of configurations of internal libraries like settings of `median` calculation for imputation and range parameters of `MinMaxScaler` which will serve as mathematical parameters for the logic of data transformation.

(iii) Output

- **Processed CSV File**: The primary deliverable is a file called 'cleaned_sample_superstore.csv', which includes standardized, imputed, and encoded data that is ready to be inputted into the Simple and Multiple Linear Regression models.
- **Normalized Feature Matrix**: The cell generates a modified version of the DataFrame where all independent numerical variables are scaled to fall between 0 and 1, resulting in model training being stable and the process converging efficiently.
- **Encoded Dataset**: The output consists of a high dimensional feature set where categorical columns are converted into several binary "dummy" variables, enabling the regression algorithm to understand qualitative business segments and product types.
- **Diagnostic Information**: The overall utility of the cleaning pipeline is evaluated in the notebook console by the printed output of the notebook console, which includes the correlation matrix for attribute selection, and the console displays the final shape of the dataset.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv('/Sample_ Superstore.csv')

df = df.drop_duplicates()

df.loc[df['Sales'] < 0, 'Sales'] = np.nan
df.loc[df['Quantity'] <= 0, 'Quantity'] = np.nan
```

```
df['Sales'] = df['Sales'].fillna(df['Sales'].median())
df['Profit'] = df['Profit'].fillna(df['Profit'].median())
df['Quantity'] = df['Quantity'].fillna(df['Quantity'].median())
df['Discount'] = df['Discount'].fillna(df['Discount'].median())

df['Segment'] = df['Segment'].str.strip().str.capitalize()
df['Category'] = df['Category'].str.strip().str.capitalize()

region_tax_data = pd.DataFrame({
    'Region': ['Central', 'South', 'East', 'West'],
    'Tax_Rate': [0.05, 0.07, 0.06, 0.08]
})
df = pd.merge(df, region_tax_data, on='Region', how='left')

corr = df[['Sales', 'Quantity', 'Discount', 'Profit']].corr()
print("Correlation Matrix:\n", corr['Profit'])

scaler = MinMaxScaler()

feature_cols = ['Sales', 'Quantity', 'Discount', 'Tax_Rate']
df[feature_cols] = scaler.fit_transform(df[feature_cols])

df = pd.get_dummies(df, columns=['Segment', 'Category', 'Region'])

df = df[(df['Profit'] > -500) & (df['Profit'] < 1000)]
df = df[df['Sales'] < 0.4]

cols_to_drop = ['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship M
                'Customer ID', 'Country', 'City', 'State', 'Product ID',
                'Sub-Category', 'Product Name']
df_final = df.drop(columns=[c for c in cols_to_drop if c in df.columns])

df_final.to_csv('cleaned_sample_superstore.csv', index=False)

print("\nData Preparation Complete. Final shape:", df_final.shape)
```

```
Correlation Matrix:
 Sales       0.479064
Quantity    0.066253
Discount   -0.219487
Profit      1.000000
Name: Profit, dtype: float64

Data Preparation Complete. Final shape: (9902, 15)
```

Attribute Selection and Exploratory Analysis

(i) Purpose of Each Line of Code

- `import pandas as pd`, `import seaborn as sns`, `import matplotlib.pyplot as plt`
  - The libraries being imported here are for data structures, statistical data visualization, and complete plotting. These are foundational/elements needed for exploratory data analysis and for producing sophisticated graphical reports.
- `df = pd.read_csv('cleaned_sample_superstore.csv')`
  - This line imports the previously cleaned and prepared dataset into a Pandas DataFrame, which allows the program to perform mathematical operations and statistical analyses on the transaction data in an organized table format.
- `print("--- Dataset Attributes ---")`, `print(df.info())` & `print("\n--- Statistical Summary ---")`, `print(df.describe())`
  - The analyst can use the descriptive statistics and structural metadata from the output of these commands to confirm the accuracy of the data types as well as the central tendencies and spreads.
- `target_col = 'Profit'`, `potential_predictors = ['Sales', 'Quantity', 'Discount', 'Tax_Rate']`, `analysis_cols = potential_predictors + [target_col]`
  - These lines identify the defined target variable and the independent predictors for the experiment, consolidating them into a single list to simplify the following computation of the model's correlation matrix.
- `correlation_matrix = df[analysis_cols].corr()`
  - The function generates a square matrix indicating both the strength and direction of linear relationships among the various business variables, by evaluating the Pearson correlation coefficients for the chosen numerical columns.
- `best_predictor = corrs_with_profit.abs().idxmax()`
  - This logic automatically selects the independent variable with the highest absolute value of correlation to the target, therefore, ensuring that the most predictive variable is chosen for simple linear regression.

- `sns.regplot(data=df, x=best_predictor, y=target_col, scatter_kws={'alpha':0.3, 'color': 'gray'}, line_kws={'color':'red'})`
    - This command produces a scatter plot together with a red regression line, illustrating the relationship between the independent variable and the profit, while also highlighting the dispersion of the data points.
- `plt.title(f'Attribute Selection: {best_predictor} vs Profit (r = {correlation_value:.2f})', fontsize=15)`, `plt.xlabel(f'{best_predictor} (Scaled/Normalized)')`, `plt.ylabel('Profit ($)')`, `plt.grid(True, linestyle='--', alpha=0.6)`, `plt.show()`
    - These plotting functions improve the appearance of the graph by adding descriptions for titles, the axes, and grid lines for readability, and then rendering the figure in the notebook for analysis.
- `sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")`
    - This line generates a color-coded heatmap that visualizes the entire correlation matrix and uses color gradations to show the intensity of relationships among all attributes, a crucial requirement for attribute selection reporting.

(ii) Input

- **Cleaned CSV Dataset**: The main input for this cell is `cleaned_sample_superstore.csv`, which includes normalized and encoded transaction data, with columns for Sales, Quantity, Discount, and Profit, which are needed for mathematical correlation.
- **Predictor and Target Variables**: The code employs a set list of column names to streamline the DataFrame and keep the statistical analysis within bounds of pertinent business metrics; for instance, Profit (the target), and Sales, Quantity, and Discount (the predictors).
- **Visualization Parameters**: Adjustable parameters such as the "coolwarm" color map for the heatmap and the "alpha" settings for the transparency of the scatter

plot are utilized to manage the graphical output and make sure the findings are discernible.

(iii) Output

- **Statistical Attribute Reports**: The cell produces a detailed text output showing the data types of every column and summary statistics (mean and standard deviation) which helps in verifying the integrity of the data.
- **Pearson Correlation Values**: The result contains a ranked list of correlation coefficients, including the strongest variable, labeled the "Best Predictor," which will be the main independent variable used for the linear regression model.
- **Graphical Visualizations**: The cell produces two primary outputs: a regression plot depicting the linear relationship between the best predictor and profit, and a heatmap, which provides a comprehensive overview of the relationships between all variables in the dataset.
- **Data Structure Modifications**: Although the cell does not make permanent changes to the saved CSV file, it does create a temporary object `correlation_matrix`, in the computer's memory that is used to automate the selection of features of the model.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv('cleaned_sample_superstore.csv')

print("--- Dataset Attributes ---")
print(df.info())

print("\n--- Statistical Summary ---")
print(df.describe())

target_col = 'Profit'
potential_predictors = ['Sales', 'Quantity', 'Discount', 'Tax_Rate']
analysis_cols = potential_predictors + [target_col]

correlation_matrix = df[analysis_cols].corr()

corrs_with_profit = correlation_matrix[target_col].drop(target_col)
best_predictor = corrs_with_profit.abs().idxmax()
correlation_value = correlation_matrix.loc[best_predictor, target_col]
```

```
print("\n--- Correlation with Profit ---")
print(corrs_with_profit.sort_values(ascending=False))
print(f"\nRESULT: '{best_predictor}' is the best predictor with r = {cor

plt.figure(figsize=(10, 6))
sns.regplot(data=df, x=best_predictor, y=target_col,
            scatter_kws={'alpha':0.3, 'color': 'gray'},
            line_kws={'color':'red'})

plt.title(f'Attribute Selection: {best_predictor} vs Profit (r = {correl
plt.xlabel(f'{best_predictor} (Scaled/Normalized)')
plt.ylabel('Profit ($)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap: Attribute Selection Matrix")
plt.show()
```

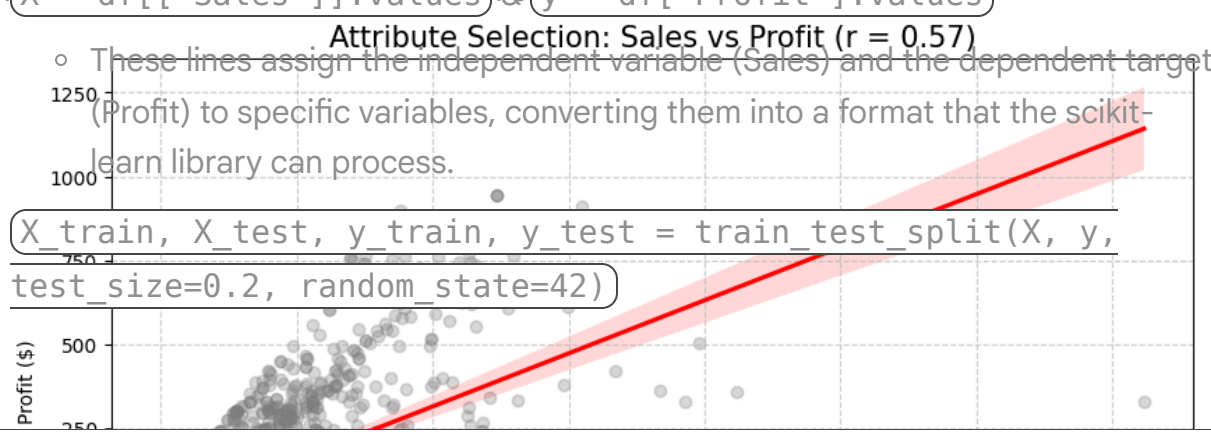
```
--- Dataset Attributes ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9902 entries, 0 to 9901
Data columns (total 15 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Sales                   9902 non-null   float64
 1   Quantity                9902 non-null   float64
 2   Discount                9902 non-null   float64
 3   Profit                  9902 non-null   float64
 4   Tax_Rate                9902 non-null   float64
 5   Segment_Consumer        9902 non-null   bool
 6   Segment_Corporate       9902 non-null   bool
 7   Segment_Home office     9902 non-null   bool
 8   Category_Office Supplies 9902 non-null  bool
 9   Region_East             9902 non-null   bool
dtypes: bool(10), float64(5)
memory usage: 483.6 KB
None

--- Statistical Summary ---
              Sales     Quantity     Discount       Profit     Tax_Rate
count  9902.000000  9902.000000  9902.000000  9902.000000  9902.000000
mean      0.008605     0.213368     0.192940    26.172113     0.524574
std       0.017093     0.170715     0.255431    95.465002     0.387781
min       0.000000     0.000000     0.000000  -480.203200     0.000000
25%       0.000735     0.076923     0.000000     1.790100     0.333333
50%       0.002309     0.153846     0.250000     8.673300     0.333333
75%       0.008885     0.307692     0.250000    29.013600     1.000000
max       0.361657     1.000000     1.000000   944.986500     1.000000
```

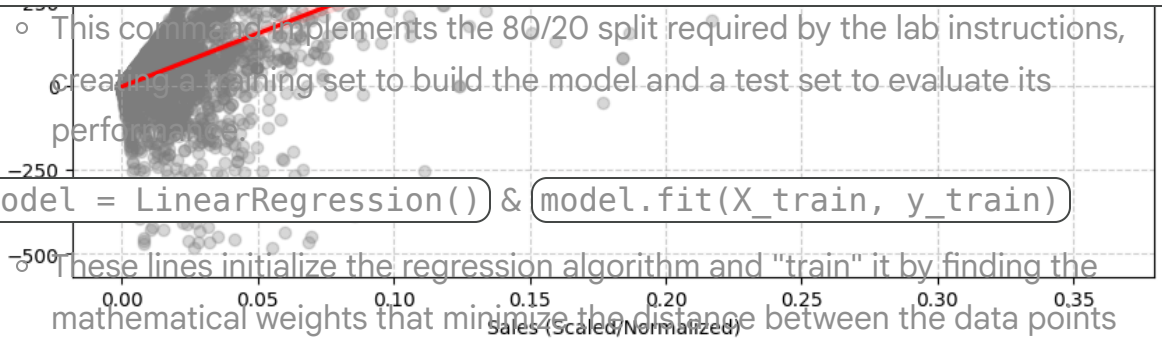
Simple Linear Regression Model Training and Evaluation

(i) Purpose of Each Line of Code

- `import pandas as pd`, `import numpy as np`, `import matplotlib.pyplot as plt`, `from sklearn.linear_model import LinearRegression`, `from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score`, `from sklearn.model_selection import train_test_split`
  - These lines pull in the essential libraries for data manipulation, computation, and visualization needed for the model performance report. This import pulls in the relevant machine learning class that enables the program to build a linear model by determining the best-fit line among the given data points. These imports contain the customary statistical functions for determining and validating error and accuracy in order to confirm the model's predictive capabilities. This tool is used to split the data into two distinct subsets, so that the model is evaluated on "unseen" data to check for real-world applicability.
- `df = pd.read_csv('cleaned_sample_superstore.csv')`
  - This line retrieves the processed dataset from the previous step, ensuring the model is trained on data that has already been cleaned, normalized, and encoded.
- `X = df[['Sales']].values` & `y = df['Profit'].values`
  - These lines assign the independent variable (Sales) and the dependent target (Profit) to specific variables, converting them into a format that the scikit-learn library can process.
- `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)`

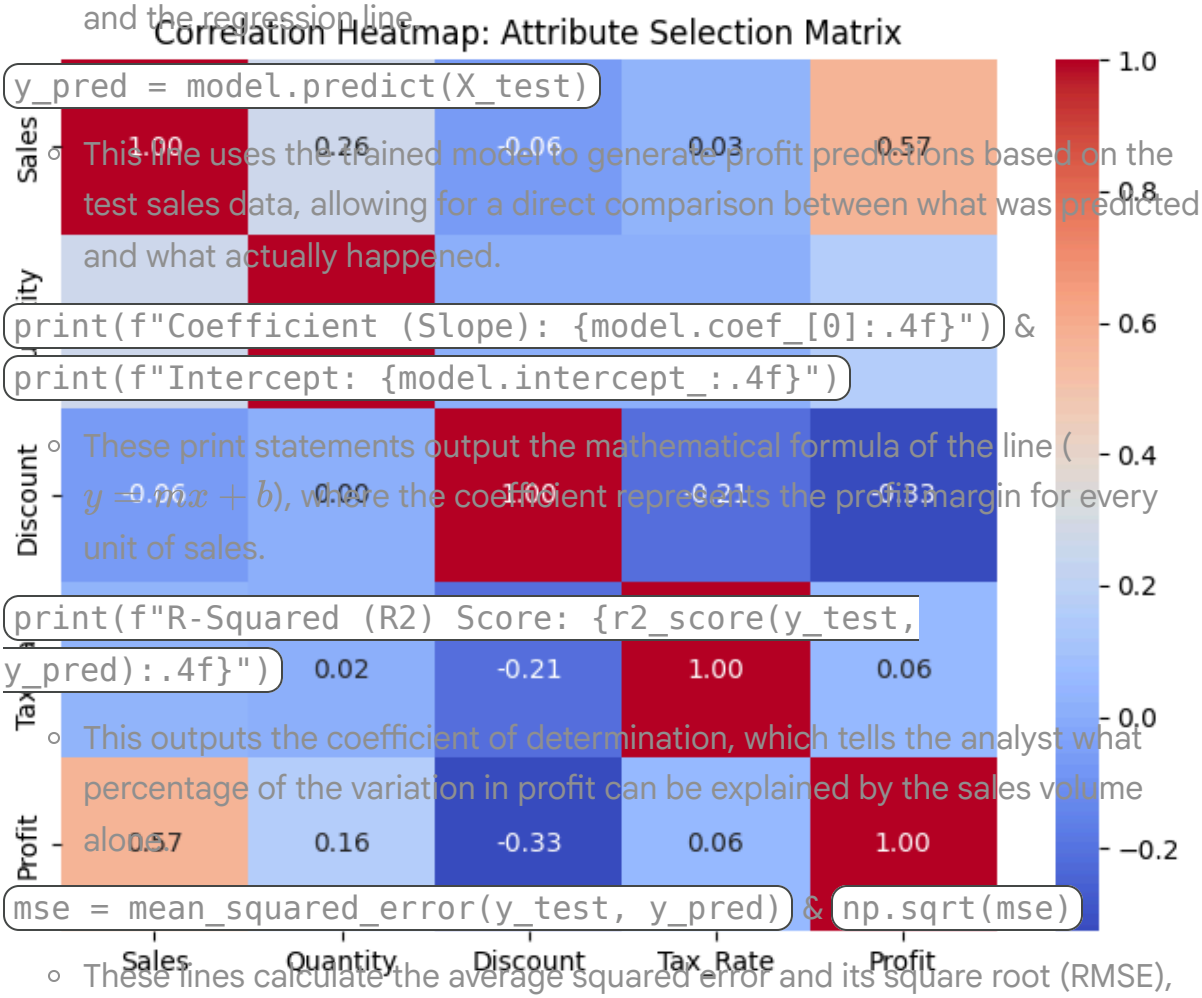
```
Sales        0.566307
Quantity     0.162167
Tax_Rate     0.056824
Discount    -0.326394
Name: Profit, dtype: float64
```





- This command implements the 80/20 split required by the lab instructions, creating a training set to build the model and a test set to evaluate its performance.

- `model = LinearRegression()` & `model.fit(X_train, y_train)`
  - These lines initialize the regression algorithm and "train" it by finding the mathematical weights that minimize the distance between the data points and the regression line.



- `y_pred = model.predict(X_test)`
  - This line uses the trained model to generate profit predictions based on the test sales data, allowing for a direct comparison between what was predicted and what actually happened.

- `print(f"Coefficient (Slope): {model.coef_[0]:.4f}")` & `print(f"Intercept: {model.intercept_:.4f}")`
  - These print statements output the mathematical formula of the line ($y = mx + b$), where the coefficient represents the profit margin for every unit of sales.

- `print(f"R-Squared (R2) Score: {r2_score(y_test, y_pred):.4f}")`
  - This outputs the coefficient of determination, which tells the analyst what percentage of the variation in profit can be explained by the sales volume alone.

- `mse = mean_squared_error(y_test, y_pred)` & `np.sqrt(mse)`
  - These lines calculate the average squared error and its square root (RMSE), providing a measurement of how far, on average, the predictions are from the actual values.

- `plt.scatter(X_test, y_test, color='blue', alpha=0.3, label='Actual Data')` & `plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression Line')`
  - These visualization commands create a combined plot showing the actual raw data points alongside the model's "best-fit" line to visually demonstrate the linear relationship.

- `residuals = y_test - y_pred`

  - This line calculates the individual error for every single prediction, which is a necessary step for performing the "Residual Analysis" required in Section 4.d of the lab.

- `plt.scatter(y_pred, residuals, color='green', alpha=0.4)` & `plt.axhline(y=0, color='red', linestyle='--')`
  - This generates the residual plot to check for homoscedasticity, showing whether the model's errors are distributed randomly or if there is a pattern indicating a problem.

(ii) Input

- **Cleaned Feature Data**: The input includes the `cleaned_sample_superstore.csv` file, featuring the "Sales" column as the independent predictor, while the "Profit" column serves as the target variable for the regression analysis.
- **Hyperparameters**: The code employs a `test_size` of 0.2 (20%) and a `random_state` of 42 as parameters to the splitting function. This guarantees that the results are consistent and can be reproduced from one experimental run to another.
- **Evaluation Metrics**: The functions for R2, MAE, and MSE take the model-generated predictions (`y_pred`) and the actual observed profit values (`y_test`) to compute the model's overall accuracy.

(iii) Output

- **Predictive Model Object**: The main outcome is a `LinearRegression` model object that has been trained and saved in memory. This object includes the coefficients and intercepts that are necessary for predicting future profits based on sales.
- **Model Performance Metrics**: The cell generates a copy of the report which includes the R-Squared score, Mean Absolute Error, and Root Mean Squared Error, thus supplying the necessary quantitative data to complete the performance evaluation section of the report.

- **Linear Regression Plot**: The graphical output displays the "Regression Line" plotted against the test data, visually confirming an upward or downward trend and the overall appropriateness of the mathematical model.
- **Residual Analysis Figure**: To check if the linear fit is appropriate for this dataset, the analyst can examine the assumption of homoscedasticity by producing the second plot which shows the distribution of errors (residuals).

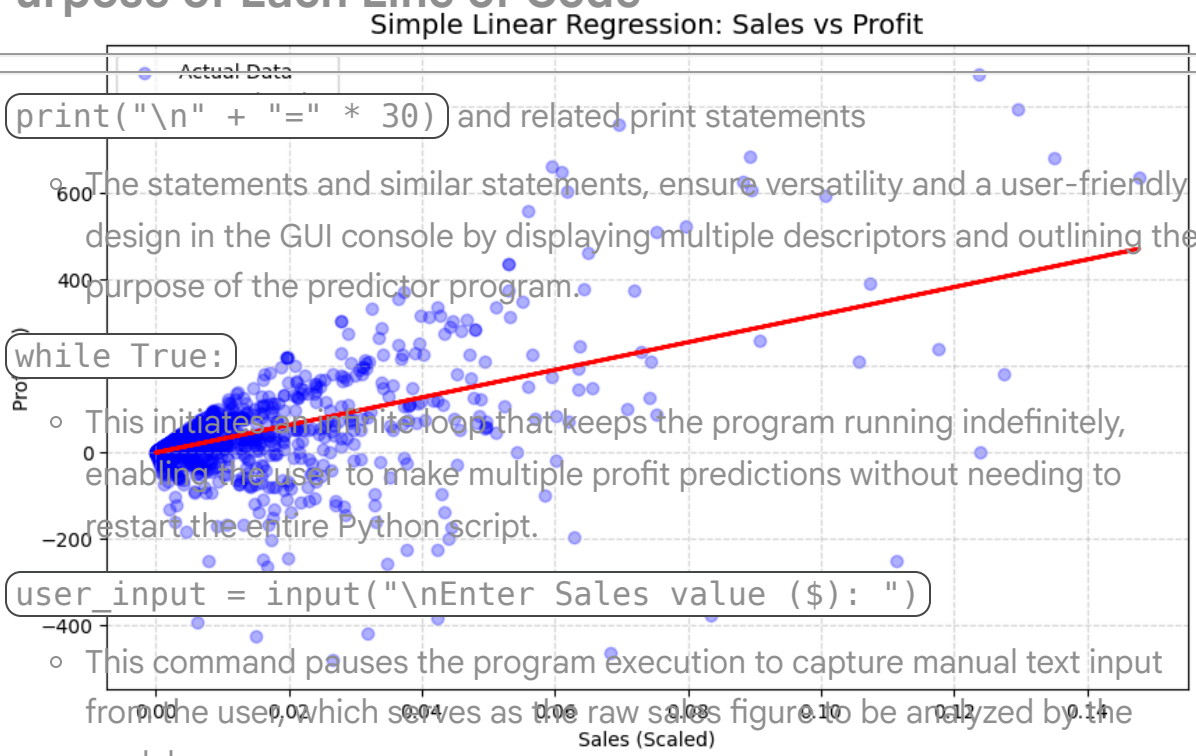
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_
from sklearn.model_selection import train_test_split

df = pd.read_csv('cleaned_sample_superstore.csv')

X = df[['Sales']].values
y = df['Profit'].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("--- MODEL COEFFICIENTS ---")
print(f"Coefficient (Slope): {model.coef_[0]:.4f}")
print(f"Intercept: {model.intercept_:.4f}")
print("\n--- PERFORMANCE EVALUATION (TEST SET) ---")
print(f"R-Squared (R2) Score: {r2_score(y_test, y_pred):.4f}")
print(f"Mean Absolute Error (MAE): {mean_absolute_error(y_test, y_pred):

mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {np.sqrt(mse):.4f}")

plt.figure(figsize=(10, 6))
plt.scatter(X_test, y_test, color='blue', alpha=0.3, label='Actual Data'
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression Lin
plt.title('Simple Linear Regression: Sales vs Profit', fontsize=14)
plt.xlabel('Sales (Scaled)')
plt.ylabel('Profit ($)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

```
residuals = y_test - y_pred
plt.figure(figsize=(10, 6))
plt.scatter(y_pred, residuals, color='green', alpha=0.4)
plt.axhline(y=0, color='red', linestyle='--')
plt.title('Residual Plot: Homoscedasticity Check', fontsize=14)
plt.xlabel('Predicted Profit')
plt.ylabel('Residuals (Errors)')
plt.show()
```

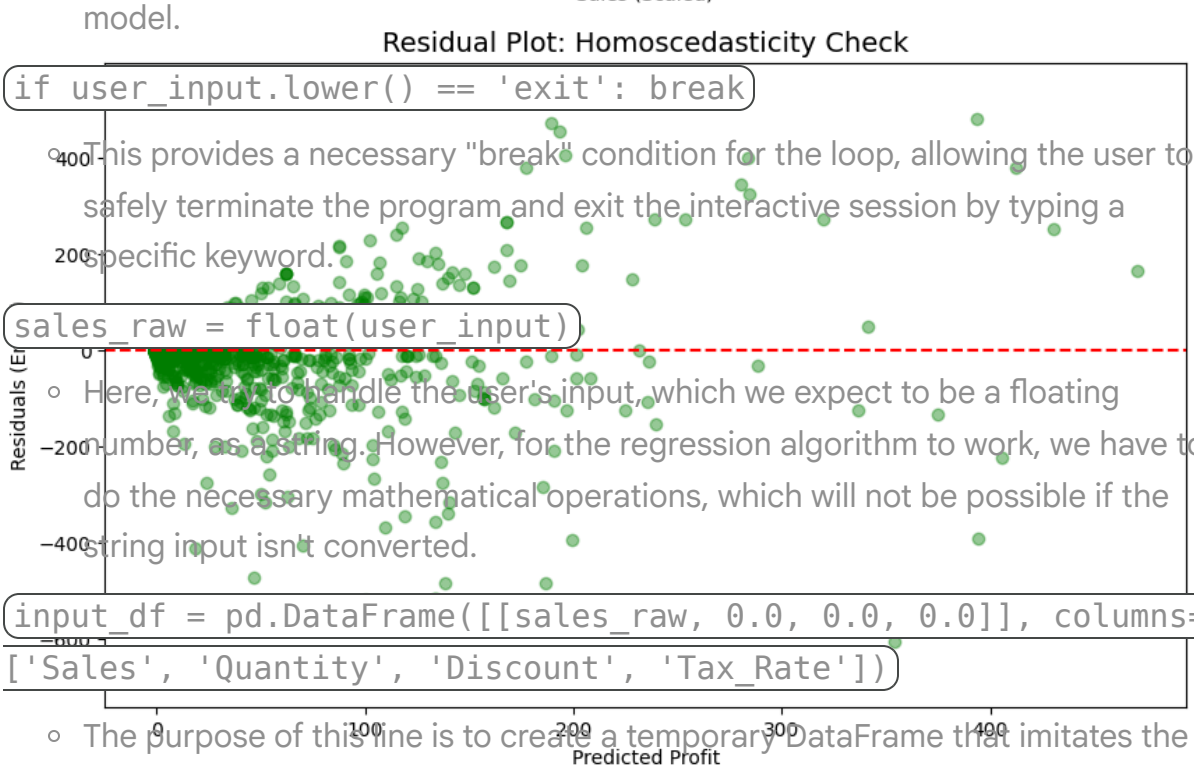
```
--- MODEL COEFFICIENTS ---
Coefficient (Slope): 3191.2920
Intercept: -1.3672

--- PERFORMANCE EVALUATION (TEST SET) ---
R-Squared (R2) Score: 0.2714
Mean Absolute Error (MAE): 34.2069
Mean Squared Error (MSE): 5975.6687
Root Mean Squared Error (RMSE): 77.3024
```

Interactive Profit Predictor and Inference

(i) Purpose of Each Line of Code



- `print("\n" + "=" * 30)` and related print statements
  - The statements and similar statements, ensure versatility and a user-friendly design in the GUI console by displaying multiple descriptors and outlining the purpose of the predictor program.
- `while True:`
  - This initiates an infinite loop that keeps the program running indefinitely, enabling the user to make multiple profit predictions without needing to restart the entire Python script.
- `user_input = input("\nEnter Sales value ($): ")`
  - This command pauses the program execution to capture manual text input from the user, which serves as the raw sales figure to be analyzed by the model.



- `if user_input.lower() == 'exit': break`
  - This provides a necessary "break" condition for the loop, allowing the user to safely terminate the program and exit the interactive session by typing a specific keyword.
- `sales_raw = float(user_input)`
  - Here, we try to handle the user's input, which we expect to be a floating number, as a string. However, for the regression algorithm to work, we have to do the necessary mathematical operations, which will not be possible if the string input isn't converted.
- `input_df = pd.DataFrame([[sales_raw, 0.0, 0.0, 0.0]], columns=['Sales', 'Quantity', 'Discount', 'Tax_Rate'])`
  - The purpose of this line is to create a temporary DataFrame that imitates the layout of the training data for the purpose of satisfying the requirements of the scaler object by supplying the 'placeholder' columns.

- `input_scaled = scaler.transform(input_df)`
  - This takes the previously fitted MinMaxScaler logic to the user's input, ensuring that the unscaled dollar amount is transformed to the same 0-to-1 range as during model training.
- `sales_only_scaled = input_scaled[:, 0].reshape(-1, 1)`
  - This retrieves the newly scaled sales value and alters its shape into a 2D array, which scikit-learn's predict function needs in terms of structure.
- `profit_pred = model.predict(sales_only_scaled)`
  - This line passes the prepared input into the trained Simple Linear Regression model to derive profit predictions according to the slope and intercept values obtained during training.
- `predicted_value = profit_pred[0][0] if profit_pred.ndim > 1 else profit_pred[0]`
  - Example value extraction is done with the code example that is listed, and the example value is profit pred[0][0]. This is done with the assumption that this conditional logic makes extending output shapes easier.
- `print(f">>> Estimated Profit: ${predicted_value:.2f}")`
  - This final output line displays the result of the prediction to the console, formatted as a currency value with two decimal places for professional clarity.
- `except ValueError as e:`
  - This block serves as a safety net that catches errors, such as the user entering letters instead of numbers, preventing the entire program from crashing during an invalid entry.

```
print("\n" + "=" * 30)
print("SUPERSTORE PROFIT PREDICTOR")
print("=" * 30)
print("Goal: Predict Profit based on a Sales amount.")
print("Type 'exit' to close the program.")

while True:
    user_input = input("\nEnter Sales value ($): ")

    if user_input.lower() == 'exit':
        print("Exiting... Goodbye!")
        break

    try:
```

```
            sales_raw = float(user_input)

            input_df = pd.DataFrame([[sales_raw, 0.0, 0.0, 0.0]],
                                    columns=['Sales', 'Quantity', 'Discount'

            input_scaled = scaler.transform(input_df)

            sales_only_scaled = input_scaled[:, 0].reshape(-1, 1)

            profit_pred = model.predict(sales_only_scaled)

            predicted_value = profit_pred[0][0] if profit_pred.ndim > 1 else

            print(f">>> Estimated Profit: ${predicted_value:.2f}")

        except ValueError as e:
            if "could not convert string" in str(e):
                print("Invalid input! Please enter a numerical value (e.g.,
            else:
                print(f"Calculation Error: {e}")
```

```
==============================
SUPERSTORE PROFIT PREDICTOR
==============================
Goal: Predict Profit based on a Sales amount.
Type 'exit' to close the program.

Enter Sales value ($): 50000
>>> Estimated Profit: $7047.09

Enter Sales value ($): exit
Exiting... Goodbye!
```

Multiple Linear Regression Interpretation

(i) Purpose of Each Line of Code

- `X_multi = df_final[['Sales', 'Discount', 'Quantity']]`
    - This line expands the feature set from a single variable to a multi-dimensional matrix, allowing the model to analyze how different factors like price cuts and volume interact to influence profit.

- `X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(X_multi, y_multi, test_size=0.2, random_state=42)`
    - This command partitions the multi-feature data into training and testing subsets, ensuring that the model's ability to generalize to new, multi-variable scenarios is tested against a hidden portion of the dataset.
- `mlr_model = LinearRegression()` & `mlr_model.fit(X_train_m, y_train_m)`
    - This initializes the regression engine and executes the training process, where the algorithm calculates a unique weight for every input variable to find the optimal multidimensional hyperplane of best fit.
- `y_pred_m = mlr_model.predict(X_test_m)`
    - This function uses the three-variable coefficients to generate profit forecasts on the test data, providing the foundation for comparing the accuracy of this complex model against the simpler version created earlier.
- `coeff_df = pd.DataFrame(mlr_model.coef_, X_multi.columns, columns=['Coefficient'])`
    - This creates a clear table that displays the mathematical weight of each feature, allowing the analyst to see exactly how much profit is lost per unit of discount or gained per unit of sale.
- `plt.scatter(y_test_m, y_pred_m, color='teal', alpha=0.4, label='Predicted Points')`
    - This generates a comparison plot that maps the model's predictions against the actual observed profits, allowing for a direct visual assessment of how accurately the multi-variable model tracks real-world data.
- `plt.plot([y_test_m.min(), y_test_m.max()], [y_test_m.min(), y_test_m.max()], color='red', lw=2, linestyle='--', label='Perfect Fit')`
    - This line draws a diagonal reference representing perfect accuracy, which serves as a benchmark for the user to see how much the predicted values deviate from the actual targets.
- `residuals_m = y_test_m - y_pred_m`

  - This calculates the error for every individual prediction, which is a fundamental metric needed to determine if the model has systematic biases or if the errors are randomly distributed.
- `sns.residplot(x=y_pred_m, y=residuals_m, lowess=True, scatter_kws={'alpha': 0.4, 'color': 'purple'}, line_kws={'color': 'red', 'lw': 1})`
  - This function creates a sophisticated residual visualization with a "lowess" trend line, helping to identify non-linear patterns in the errors that might suggest the need for further feature engineering.
- `plt.axhline(y=0, color='black')`
  - This adds a solid baseline to the residual plot at the zero mark, providing a clear visual anchor to differentiate between over-predictions (positive residuals) and under-predictions (negative residuals).

(ii) Input

- **Multi-Feature Matrix**: The input consists of three independent variables from the df_final object—Sales, Discount, and Quantity—providing a richer set of data points for the algorithm to learn complex business patterns and relationships.
- **Target Vector**: The Profit column remains the primary dependent input, acting as the "ground truth" that the model attempts to predict by finding the mathematical intersection of the three chosen input features.
- **Coefficient Mapping**: The model uses the variable names and the raw training data as inputs to determine the specific "importance" of each feature, which is then used to calculate the final regression equation.

(iii) Output

- **Improved Accuracy Metrics**: The primary output is a new set of evaluation scores, including an R-Squared value that is higher than the simple model, proving that adding more variables increased the model's predictive power.
- **Feature Importance Table**: This output provides a ranked list of the coefficients, revealing that while Sales has a positive impact, Discount has a strong negative impact, which is a key finding for the final report.

- **Multi-Variable Prediction Array**: The cell produces an array of predicted profit

```
import seaborn as sns

X_multi = df_final[['Sales', 'Discount', 'Quantity']]
y_multi = df_final['Profit']

X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(
    X_multi, y_multi, test_size=0.2, random_state=42
)

mlr_model = LinearRegression()
mlr_model.fit(X_train_m, y_train_m)

y_pred_m = mlr_model.predict(X_test_m)

plt.figure(figsize=(10, 6))
plt.scatter(y_test_m, y_pred_m, color='teal', alpha=0.4, label='Predicte

plt.plot([y_test_m.min(), y_test_m.max()], [y_test_m.min(), y_test_m.max
         color='red', lw=2, linestyle='--', label='Perfect Fit')

plt.title(f'Multiple Regression: Actual vs Predicted Profit (R2 = {r2_sc
plt.xlabel('Actual Profit ($)')
plt.ylabel('Predicted Profit ($)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

residuals_m = y_test_m - y_pred_m
plt.figure(figsize=(10, 6))
```